🔷 COMANDOS .NET - GUIA COMPLETO

🚀 COMANDOS BÁSICOS DO DOTNET CLI

Informação e Versão

```bash
# Informações detalhadas do ambiente .NET
dotnet --info

# Versão do .NET SDK instalada
dotnet --version

# Ajuda geral
dotnet -h
dotnet --help

# Ajuda específica por comando
dotnet new -h
```

```bash
dotnet build --help
```

🏗️ CRIAÇÃO E ESTRUTURA DE PROJETOS

Templates e Soluções

```bash
# Listar todos os templates disponíveis
dotnet new list

# Filtrar templates por tipo
dotnet new list web
dotnet new list api
dotnet new list console
```

```bash
# Criar nova solution
dotnet new sln -n MinhaSolution
dotnet new sln

# Criar projetos
dotnet new webapi -o API --controllers
dotnet new webapi -o API --use-controllers
dotnet new classlib -o Core
dotnet new classlib -o Infrastructure
dotnet new console -o ConsoleApp
dotnet new mvc -o WebApp
dotnet new blazorserver -o BlazorApp
```

```
dotnet new xunit -o Tests
```

Gerenciamento de Solution

```bash
# Adicionar projetos à solution
dotnet sln add API
dotnet sln add Core
dotnet sln add Infrastructure
dotnet sln add Tests

# Adicionar múltiplos projetos
dotnet sln add **/*.csproj

# Listar projetos na solution
dotnet sln list

# Remover projeto da solution
```

```
dotnet sln remove API
```

🔗 DEPENDÊNCIAS E REFERÊNCIAS

Referências entre Projetos

```bash
```

```bash
# Na pasta API/
dotnet add reference ../Infrastructure
dotnet add reference ../Core

# Na pasta Infrastructure/
dotnet add reference ../Core

# Na pasta Tests/
dotnet add reference ../API
dotnet add reference ../Core

# Listar referências
dotnet list reference
```

Pacotes NuGet

```bash
# Adicionar pacote NuGet
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
dotnet add package Swashbuckle.AspNetCore

# Adicionar versão específica
dotnet add package Newtonsoft.Json --version 13.0.3

# Listar pacotes instalados
dotnet list package

# Atualizar pacotes
dotnet update package

# Remover pacote
dotnet remove package Newtonsoft.Json
```

🏗️ COMPILAÇÃO E EXECUÇÃO

Build e Restore

bash

```bash
# Restaurar dependências
dotnet restore

# Build do projeto/solution
dotnet build
dotnet build --configuration Release
dotnet build --verbosity detailed

# Build específico para runtime
dotnet build --runtime linux-x64

# Clean build
dotnet clean
```

bash

```bash
dotnet clean && dotnet build
```

Execução

bash

```bash
# Executar projeto
dotnet run
dotnet run --project API

# Executar com arguments
dotnet run -- arg1 arg2

# Executar em modo Release
dotnet run --configuration Release

# Watch mode (desenvolvimento)
dotnet watch run
dotnet watch run --project API

# Watch com hot reload
```

bash

```bash
dotnet watch run --hot-reload
```

📊 COMANDOS AVANÇADOS

Testes

```bash
# Executar testes
dotnet test
dotnet test --verbosity normal
dotnet test --logger "console;verbosity=detailed"

# Executar testes específicos
dotnet test --filter "Category=Integration"
dotnet test --filter "FullyQualifiedName~MyTestClass"

# Code coverage
dotnet test --collect:"XPlat Code Coverage"
```

Publicação (Publish)

```bash
# Publicar para pasta
dotnet publish
dotnet publish -o ./publish
dotnet publish --configuration Release

# Publicar self-contained
dotnet publish --self-contained true --runtime linux-x64

# Publicar single file
dotnet publish --self-contained true --runtime win-x64
-p:PublishSingleFile=true
```

Entity Framework Core

```bash
# Instalar EF Core CLI
```

```
dotnet tool install --global dotnet-ef

# Migrations
dotnet ef migrations add InitialCreate
dotnet ef migrations add AddUserTable
dotnet ef migrations remove

# Database update
dotnet ef database update
dotnet ef database update InitialCreate

# Gerar script SQL
dotnet ef migrations script
dotnet ef migrations script --idempotent

# Context info
dotnet ef dbcontext info
```

```
dotnet ef dbcontext scaffold
```

🎯 EXEMPLO DE FLUXO COMPLETO

Criando uma Arquitetura Completa

```
bash
```

```
# 1. Criar estrutura de pastas
mkdir MyEnterpriseApp
cd MyEnterpriseApp

# 2. Criar solution
dotnet new sln -n EnterpriseApp

# 3. Criar projetos
dotnet new webapi -o src/API --use-controllers
dotnet new classlib -o src/Core
dotnet new classlib -o src/Infrastructure
dotnet new xunit -o tests/UnitTests
dotnet new xunit -o tests/IntegrationTests

# 4. Adicionar projetos à solution
dotnet sln add src/API
dotnet sln add src/Core
dotnet sln add src/Infrastructure
dotnet sln add tests/UnitTests
```

```
dotnet sln add tests/IntegrationTests

# 5. Configurar dependências
cd src/API
dotnet add reference ../Core
dotnet add reference ../Infrastructure

cd ../Infrastructure
dotnet add reference ../Core

cd ../../tests/UnitTests
dotnet add reference ../../src/Core

cd ../IntegrationTests
dotnet add reference ../../src/API
dotnet add reference ../../src/Infrastructure

# 6. Adicionar pacotes NuGet
cd ../../src/API
dotnet add package Swashbuckle.AspNetCore
dotnet add package Microsoft.EntityFrameworkCore.Design

cd ../Infrastructure
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

# 7. Build e execução
cd ../..
dotnet restore
dotnet build
```

```
dotnet run --project src/API
```

🔧 COMANDOS DE MANUTENÇÃO

Limpeza e Otimização

```
bash
```

```
# Limpar caches
dotnet nuget locals all --clear
dotnet clean

# Verificar saúde do projeto
dotnet format
```

```
dotnet format --verify-no-changes

# Analisar dependências
dotnet list package --outdated
dotnet list package --include-transitive

# Verificar vulnerabilidades
```

```
dotnet list package --vulnerable
```

Global Tools

```
bash
```

```
# Listar tools instaladas
dotnet tool list -g

# Instalar tools úteis
dotnet tool install -g dotnet-ef
dotnet tool install -g dotnet-reportgenerator-globaltool
dotnet tool install -g dotnet-outdated-tool

# Executar tools
dotnet outdated
```

```
reportgenerator -reports:coverage.cobertura.xml
-targetdir:coveragereport
```

🐳 DOCKER PARA .NET

Dockerfile para .NET

```
dockerfile
```

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["API/API.csproj", "API/"]
COPY ["Core/Core.csproj", "Core/"]
COPY ["Infrastructure/Infrastructure.csproj", "Infrastructure/"]
```

```
RUN dotnet restore "API/API.csproj"
COPY . .
WORKDIR "/src/API"
RUN dotnet build "API.csproj" -c Release -o /app/build

# Publish stage
FROM build AS publish
RUN dotnet publish "API.csproj" -c Release -o /app/publish

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=publish /app/publish .

      ENTRYPOINT ["dotnet", "API.dll"]
```

Docker Compose

```
      yaml
```

```yaml
version: '3.8'

services:
  api:
    build:
      context: .
      dockerfile: API/Dockerfile
    ports:
      - "5000:80"
      - "5001:443"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      -
ConnectionStrings__DefaultConnection=Server=db;Database=MyApp;User=sa;Password=You
rPassword123;
    depends_on:
      - db

  db:
    image: mcr.microsoft.com/mssql/server:2019-latest
    environment:
      SA_PASSWORD: "YourPassword123!"
      ACCEPT_EULA: "Y"
    ports:

            - "1433:1433"
```

📝 SCRIPTS ÚTEIS

Script de Desenvolvimento

bash

```bash
#!/bin/bash
# dev.sh

echo "🔷 Iniciando ambiente de desenvolvimento .NET"

# Restaurar dependências
echo "Restaurando dependências..."
dotnet restore

# Build
echo "Fazendo build..."
dotnet build

# Executar testes
echo "Executando testes..."
dotnet test

# Executar aplicação com watch
echo "Iniciando aplicação com watch..."
dotnet watch run --project src/API
```

Script de Deploy

bash

```bash
#!/bin/bash
# deploy.sh

set -e  # Para em caso de erro

echo "🚀 Iniciando deploy..."

# Build Release
dotnet build --configuration Release
```

```
# Executar testes
dotnet test --configuration Release --no-build

# Publicar
dotnet publish --configuration Release --output ./publish

# Database update (se necessário)
dotnet ef database update --project src/Infrastructure --startup-project src/API

    echo "✅ Deploy concluído!"
```

Script de Migração do Banco

```
bash

#!/bin/bash
# migrate.sh

echo "🔄 Executando migrações do banco de dados..."

# Criar migração (se necessário)
# dotnet ef migrations add $1 --project src/Infrastructure --startup-project
src/API

# Aplicar migrações
dotnet ef database update --project src/Infrastructure --startup-project src/API

    echo "✅ Migrações aplicadas com sucesso!"
```

🛠️ COMANDOS PARA DIFERENTES TIPOS DE PROJETOS

ASP.NET Core Web API

```
bash

dotnet new webapi -o MyApi --use-controllers
```

```
cd MyApi
dotnet add package Swashbuckle.AspNetCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet run --urls="https://localhost:5001;http://localhost:5000"
```

Blazor Server

```
bash
```

```
dotnet new blazorserver -o MyBlazorApp
cd MyBlazorApp
```

```
dotnet watch run
```

Console Application

```
bash
```

```
dotnet new console -o MyConsoleApp
cd MyConsoleApp
dotnet run
```

```
dotnet publish -r win-x64 -c Release --self-contained true
```

Class Library

```
bash
```

```
dotnet new classlib -o MyLibrary
cd MyLibrary
dotnet add package Newtonsoft.Json
```

```
dotnet build
```

XUnit Test Project

bash

```
dotnet new xunit -o MyTests
cd MyTests
dotnet add reference ../MyLibrary
dotnet add package Moq
dotnet add package FluentAssertions
```

bash

```
dotnet test
```

🔍 TROUBLESHOOTING

Problemas Comuns

bash

```
# SDK não encontrado
dotnet --list-sdks
dotnet --list-runtimes

# Clean e rebuild
dotnet clean
dotnet restore
dotnet build

# Cache de pacotes corrompido
dotnet nuget locals all --clear

# Conflito de versões
dotnet --version
# Verificar global.json para versão específica

# Erro de runtime
dotnet --info
```

bash

```
# Verificar se runtime está instalado
```

Debug

bash

```
# Build com símbolos de debug
dotnet build --configuration Debug

# Executar com debug
dotnet run --configuration Debug

# Log detalhado
dotnet build --verbosity diagnostic

      dotnet run --verbosity detailed
```

Este guia cobre todos os comandos essenciais para desenvolvimento .NET profissional! 🔷